Chapitre 5 : Transitions, Animations et Interactions avec D3.js

Par Robert DIASSÉ

Objectifs :

1. Apprendre à utiliser les **transitions** pour animer les propriétés des éléments SVG.
2. Découvrir comment créer des **animations avancées**.
3. Implémenter des **interactions utilisateur** simples et complexes.

1. Transitions dans D3.js

Une **transition** dans D3.js est un processus qui permet de changer progressivement les propriétés d'un élément SVG (comme la position, la couleur, ou la taille) d'une valeur initiale à une valeur finale sur une certaine période de temps. Elle permet d'ajouter une dynamique visuelle aux graphiques en rendant les changements plus fluides et moins brusques.

Propriétés des Transitions :

- `duration(ms)` : Définit la durée de la transition en millisecondes. Plus cette valeur est élevée, plus le changement est progressif. Exemple : `.duration(1000)` pour 1 seconde.
- `ease(easeType)` : Permet de choisir l'effet de l'accélération ou de la décélération. L'option `ease` contrôle la vitesse de transition en fonction du temps. Par exemple, `d3.easeLinear` (accélération/décélération constante), `d3.easeBounce` (effet rebond), `d3.easeCubic` (accélération/décélération cubique).
- `delay(ms)` : Applique un délai avant le début de la transition. Exemple : `.delay(500)` retardera la transition de 0,5 seconde.
- `attr()` : Permet de modifier les attributs des éléments SVG au cours de la transition. Par exemple, `attr("x", 100)` modifie la position horizontale d'un élément.
- `style()` : Utilisé pour animer les propriétés CSS des éléments (comme la couleur ou l'opacité).
- `transition()` : L'appel à `.transition()` indique qu'une transition doit avoir lieu sur un élément. Elle peut être enchaînée avec d'autres méthodes pour appliquer des transformations, des styles, etc.

Exemple de Transition (Hauteur des barres)

```
<svg width="500" height="300"></svg>
<script>
  const data = [10, 20, 30, 40, 50];
  const svg = d3.select("svg");

  const xScale = d3.scaleBand()
    .domain(data.map((d, i) => i))
    .range([0, 500])
    .padding(0.1);

  const yScale = d3.scaleLinear()
    .domain([0, d3.max(data)])
    .range([300, 0]);

  svg.selectAll("rect")
    .data(data)
    .enter()
    .append("rect")
    .attr("x", (d, i) => xScale(i))
    .attr("y", 300) // Initialement en bas
    .attr("width", xScale.bandwidth())
    .attr("height", 0) // Hauteur initiale nulle
    .attr("fill", "teal")
    .transition() // Début de la transition
    .duration(1000) // Durée de 1 seconde
    .ease(d3.easeBounce) // Effet de rebond
    .attr("y", d => yScale(d))
    .attr("height", d => 300 - yScale(d));
</script>
```

---

2. Animation avancée dans D3.js

Les animations avancées dans D3.js permettent de manipuler non seulement les propriétés des éléments, mais aussi de créer des mouvements complexes (par exemple, des mouvements circulaires ou des trajectoires non linéaires). Ces animations peuvent être utilisées pour simuler des objets en mouvement, des trajectoires courbes, ou des changements de style d'éléments SVG en temps réel.

Propriétés des Animations :

- `attrTween()` : Cette fonction permet de définir des animations basées sur des interpolations personnalisées. Elle est particulièrement utile pour animer des transformations comme les rotations, les translations, ou les mouvements le long de courbes complexes.
- `duration()` : Comme dans les transitions, `duration` détermine combien de temps une animation prendra pour s'exécuter. Elle est souvent utilisée en combinaison avec `attrTween()`.
- `ease()` : Permet de contrôler le type d'animation. Par exemple, `easeLinear` pour une animation linéaire et `easeElastic` pour un effet plus dynamique.

- `d3.interval()` : Permet de répéter l'animation à intervalles réguliers. Cela est utilisé pour les animations continues comme la rotation ou les mouvements périodiques.

Exemple d'Animation Avancée (Mouvement Circulaire)

```
<svg width="500" height="500"></svg>
<script>
  const svg = d3.select("svg");
  const circle = svg.append("circle")
    .attr("cx", 250)
    .attr("cy", 150)
    .attr("r", 10)
    .attr("fill", "red");

  const radius = 100;

  d3.interval(() => {
    circle.transition()
      .duration(2000)
      .attrTween("transform", () => {
        const interpolateAngle = d3.interpolate(0, Math.PI * 2);
        return t => {
          const angle = interpolateAngle(t);
          const x = 250 + radius * Math.cos(angle);
          const y = 250 + radius * Math.sin(angle);
          return `translate(${x - 250},${y - 250})`;
        };
      });
  }, 2000); // Animation toutes les 2 secondes
</script>
```

---

3. Interactions utilisateur avec D3.js

Les interactions utilisateur permettent d'ajouter des fonctionnalités interactives à une visualisation. Cela permet à l'utilisateur de modifier la visualisation en temps réel, comme lors du survol d'un élément, du zoom ou du déplacement d'une vue. D3.js facilite l'ajout de telles interactions avec des événements tels que `mouseover`, `click`, ou `zoom`.

Propriétés des Interactions :

- `on("event", callback)` : Permet de définir une fonction de rappel pour un événement particulier (par exemple, `mouseover`, `click`, etc.). Cette fonction est appelée lorsque l'événement se produit sur un élément.

- `d3.select(this)` : Dans les événements, `this` fait référence à l'élément sur lequel l'événement est déclenché. Cela permet de manipuler cet élément directement.

- `zoom()` : Cette méthode permet d'ajouter un zoom et un panoramique à un graphique. Elle permet de manipuler l'échelle et la position de la vue d'un graphique en fonction des actions de l'utilisateur.

- `scaleExtent()` : Définit l'échelle de zoom minimale et maximale. Par exemple, `[1, 5]` permet de zoomer entre 1x et 5x.

Exemple d'Interaction (Changement de couleur au survol)

```
<svg width="500" height="300"></svg>
<script>
  const data = [10, 20, 30, 40, 50];
  const svg = d3.select("svg");

  const xScale = d3.scaleBand()
    .domain(data.map((d, i) => i))
    .range([0, 500])
    .padding(0.1);

  const yScale = d3.scaleLinear()
    .domain([0, d3.max(data)])
    .range([300, 0]);

  svg.selectAll("rect")
    .data(data)
    .enter()
    .append("rect")
    .attr("x", (d, i) => xScale(i))
    .attr("y", d => yScale(d))
    .attr("width", xScale.bandwidth())
    .attr("height", d => 300 - yScale(d))
    .attr("fill", "steelblue")
    .on("mouseover", function () {
      d3.select(this).transition()
        .duration(200)
        .attr("fill", "orange");
    })
    .on("mouseout", function () {
      d3.select(this).transition()
        .duration(200)
        .attr("fill", "steelblue");
    });
</script>
```

Conclusion

D3.js permet de créer des visualisations dynamiques et interactives grâce aux transitions, aux animations et aux interactions. Ces concepts peuvent être combinés pour produire des graphiques riches et interactifs. N'oublie pas de maîtriser l'utilisation des événements pour rendre les visualisations plus engageantes.